DBMS – ADVANCED SQL QUERY SYSTEM

Project Title: MAX – Sustainable Products E-Commerce Order Management System

1. Subqueries and Nested Queries

Find orders whose amount is greater than the average order amount.

```
SQL> SELECT Order_ID,
  2         Order_Amount
  3  FROM (
  4      SELECT Order_ID,
  5             SUM(Subtotal) AS Order_Amount
  6      FROM Order_Details
  7      GROUP BY Order_ID
  8  )
  9  WHERE Order_Amount > (
 10      SELECT AVG(Order_Amount)
 11      FROM (
 12          SELECT SUM(Subtotal) AS Order_Amount
 13          FROM Order_Details
 14          GROUP BY Order_ID
 15      )
 16  );

  ORDER_ID ORDER_AMOUNT
---------- ------------
      1001      5000.00
      1002      4500.00

SQL> SELECT Product_ID,
  2         Product_Name,
  3         Price
  4  FROM Product
  5  WHERE Price > (
  6      SELECT AVG(Price)
  7      FROM Product
  8  )
  9  ORDER BY Price DESC;

PRODUCT_ID PRODUCT_NAME                         PRICE
---------- ------------------------------ ----------
       105 Sports Shoes                         2499
```

2. Find Products Above Average Price

Display products whose price is greater than the average product price.

```
SQL> SELECT Product_ID,
  2         Product_Name,
  3         Price
  4  FROM Product
  5  WHERE Price > (
  6      SELECT AVG(Price)
  7      FROM Product
  8  )
  9  ORDER BY Price DESC;

PRODUCT_ID PRODUCT_NAME                    PRICE
---------- ------------------------------ ----------
       105 Sports Shoes                         2499
       104 Women Jeans                          1599
       118 Bamboo Chair                         1500
       102 Men Shirt                            1299
       115 Steel Bottle                         1200
```

2.1 Products Below Average Price

```
SQL> SELECT Product_ID,
  2         Product_Name,
  3         Price
  4  FROM Product
  5  WHERE Price < (
  6      SELECT AVG(Price)
  7      FROM Product
  8  )
  9  ORDER BY Price ASC;

PRODUCT_ID PRODUCT_NAME                         PRICE
---------- ------------------------------ ----------
       117 Paper Pen                              50
       113 Bamboo Toothbrush                     100
       112 Eco Notebook                          150
       109 Paper Bags                            150
       110 Wooden Brush                          200
       119 Reusable Cup                          200
       114 Jute Bag                              250
       107 Eco Bag                               250
       101 Women Kurti                           300
       111 Cotton Bag                            300
       106 Cotton Bag                            300
       116 Cotton Towel                          350
       108 Bamboo Bottle                         350
       103 Kids T-Shirt                          499

14 rows selected.
```

3. Identify Customers with Maximum Purchases

Step 1 – Calculate Customer-wise Purchase

```
SQL> SELECT C.Customer_ID,
  2         C.Customer_Name,
  3         SUM(D.Subtotal) AS Total_Purchase
  4  FROM Customer C
  5  JOIN Orders O
  6  ON C.Customer_ID = O.Customer_ID
  7  JOIN Order_Details D
  8  ON O.Order_ID = D.Order_ID
  9  GROUP BY C.Customer_ID, C.Customer_Name
 10  ORDER BY Total_Purchase DESC;

CUSTOMER_ID CUSTOMER_NAME        TOTAL_PURCHASE
----------- -------------------- --------------
          1 Keerthi                    5000.00
          2 Priya                      4500.00
          3 Divya                      1500.00
```

Step 2 – Find the Maximum Purchase

```
SQL> SELECT MAX(Total_Purchase) AS Maximum_Purchase
  2  FROM (
  3      SELECT O.Customer_ID,
  4             SUM(D.Subtotal) AS Total_Purchase
  5      FROM Orders O
  6      JOIN Order_Details D
  7      ON O.Order_ID = D.Order_ID
  8      GROUP BY O.Customer_ID
  9  );

MAXIMUM_PURCHASE
----------------
         5000.00
```

3.1 Display Customer Details with Maximum Purchase

```
SQL> SELECT C.Customer_ID,
  2         C.Customer_Name,
  3         SUM(D.Subtotal) AS Total_Purchase
  4  FROM Customer C
  5  JOIN Orders O
  6  ON C.Customer_ID = O.Customer_ID
  7  JOIN Order_Details D
  8  ON O.Order_ID = D.Order_ID
  9  GROUP BY C.Customer_ID, C.Customer_Name
 10  HAVING SUM(D.Subtotal) = (
 11      SELECT MAX(Total_Purchase)
 12      FROM (
 13          SELECT O.Customer_ID,
 14                 SUM(D.Subtotal) AS Total_Purchase
 15          FROM Orders O
 16          JOIN Order_Details D
 17          ON O.Order_ID = D.Order_ID
 18          GROUP BY O.Customer_ID
 19      )
 20  );

CUSTOMER_ID CUSTOMER_NAME        TOTAL_PURCHASE
----------- -------------------- --------------
          1 Keerthi                    5000.00
```

4. Complex Business Queries

4.1 Customers Who Placed Orders Above Average Order Value

Display customers whose order amount is greater than the average order amount

```
SQL> SELECT C.Customer_ID,
  2         C.Customer_Name,
  3         O.Order_ID,
  4         SUM(D.Subtotal) AS Order_Amount
  5  FROM Customer C
  6  JOIN Orders O
  7  ON C.Customer_ID = O.Customer_ID
  8  JOIN Order_Details D
  9  ON O.Order_ID = D.Order_ID
 10  GROUP BY C.Customer_ID, C.Customer_Name, O.Order_ID
 11  HAVING SUM(D.Subtotal) > (
 12      SELECT AVG(Order_Amount)
 13      FROM (
 14          SELECT Order_ID,
 15                 SUM(Subtotal) AS Order_Amount
 16          FROM Order_Details
 17          GROUP BY Order_ID
 18      )
 19  )
 20  ORDER BY Order_Amount DESC;

CUSTOMER_ID CUSTOMER_NAME          ORDER_ID ORDER_AMOUNT
----------- -------------------- ---------- ------------
          1 Keerthi                    1001      5000.00
          2 Priya                      1002      4500.00
```

5. Find Products Whose Sales Are Above Average

Find products whose total sales are greater than the average product sales.

```
SQL> SELECT P.Product_ID,
  2         P.Product_Name,
  3         SUM(D.Subtotal) AS Total_Sales
  4  FROM Product P
  5  JOIN Order_Details D
  6  ON P.Product_ID = D.Product_ID
  7  GROUP BY P.Product_ID, P.Product_Name
  8  HAVING SUM(D.Subtotal) > (
  9      SELECT AVG(Product_Sales)
 10      FROM (
 11          SELECT SUM(Subtotal) AS Product_Sales
 12          FROM Order_Details
 13          GROUP BY Product_ID
 14      )
 15  )
 16  ORDER BY Total_Sales DESC;

PRODUCT_ID PRODUCT_NAME                    TOTAL_SALES
---------- ------------------------------ -----------
       101 Women Kurti                        3000.00
       104 Women Jeans                        2500.00
```

6. Customer Purchase Report

Display customer ID, customer name, number of orders, total purchase amount and average order value.

```
SQL> SELECT C.Customer_ID,
  2         C.Customer_Name,
  3         COUNT(DISTINCT O.Order_ID) AS Total_Orders,
  4         SUM(D.Subtotal) AS Total_Purchase,
  5         AVG(Order_Total) AS Average_Order_Amount
  6  FROM Customer C
  7  JOIN Orders O
  8  ON C.Customer_ID = O.Customer_ID
  9  JOIN Order_Details D
 10  ON O.Order_ID = D.Order_ID
 11  JOIN (
 12      SELECT Order_ID,
 13             SUM(Subtotal) AS Order_Total
 14      FROM Order_Details
 15      GROUP BY Order_ID
 16  ) OT
 17  ON O.Order_ID = OT.Order_ID
 18  GROUP BY C.Customer_ID, C.Customer_Name
 19  ORDER BY Total_Purchase DESC;

CUSTOMER_ID CUSTOMER_NAME       TOTAL_ORDERS TOTAL_PURCHASE AVERAGE_ORDER_AMOUNT
----------- ------------------- ------------ -------------- --------------------
          1 Keerthi                        1        5000.00              5000.00
          2 Priya                          1        4500.00              4500.00
          3 Divya                          1        1500.00              1500.00
```

7. Advanced Product Sales Report

Display number of orders, total quantity sold, total sales and average selling price for each product.

```
SQL> SELECT P.Product_ID,
  2         P.Product_Name,
  3         COUNT(DISTINCT D.Order_ID) AS Total_Orders,
  4         SUM(D.Quantity) AS Total_Quantity,
  5         SUM(D.Subtotal) AS Total_Sales,
  6         AVG(D.Unit_Price) AS Average_Selling_Price
  7  FROM Product P
  8  JOIN Order_Details D
  9  ON P.Product_ID = D.Product_ID
 10  GROUP BY P.Product_ID, P.Product_Name
 11  ORDER BY Total_Sales DESC;

PRODUCT_ID PRODUCT_NAME                    TOTAL_ORDERS TOTAL_QUANTITY TOTAL_SALES AVERAGE_SELLING_PRICE
---------- ------------------------------ ------------ -------------- ----------- --------------------
       101 Women Kurti                               1              2    3000.00              1500.00
       104 Women Jeans                               1              1    2500.00              2500.00
       102 Men Shirt                                 1              1    2000.00              2000.00
       103 Kids T-Shirt                              1              2    2000.00              1000.00
       105 Sports Shoes                              1              3    1500.00               500.00
```

8. Category-wise Advanced Sales Report

Display category-wise total quantity sold and total sales amount.

```
SQL> SELECT P.Category,
  2         COUNT(DISTINCT D.Order_ID) AS Total_Orders,
  3         SUM(D.Quantity) AS Total_Quantity,
  4         SUM(D.Subtotal) AS Total_Sales
  5  FROM Product P
  6  JOIN Order_Details D
  7  ON P.Product_ID = D.Product_ID
  8  GROUP BY P.Category
  9  ORDER BY Total_Sales DESC;

CATEGORY             TOTAL_ORDERS TOTAL_QUANTITY TOTAL_SALES
-------------------- ------------ -------------- -----------
Women                           2              3     5500.00
Men                             1              1     2000.00
Kids                            1              2     2000.00
Footwear                        1              3     1500.00
```